

PRD: Research Request Prioritization & ROI Scoring Tool

TL;DR

An internal tool for Wake Forest School of Medicine data managers, enabling automatic ranking and prioritization of incoming EHR data requests from researchers. The tool computes an ROI score for each request based on PI publication metrics, journal prestige, and grant funding data pulled from public APIs—including full Clarivate JCR access. Its core benefit: immediately surfaces higher-impact requests, saving evaluation time and driving transparent, defensible prioritization. Users simply upload the REDCap Excel file; all scoring and enrichment happens seamlessly in the background, with results delivered when ready. Users may now also upload a Trac Excel file alongside the REDCap export. The Trac file provides team hours spent per request and supplies the "investment" side of the ROI calculation. The tool calculates and displays two score components to users: a Weighted Return Score (raw return-side metrics: citation, journal, funding, and research output) and an Effort-Adjusted ROI Score (the Return Score normalized by historical hours invested using Trac totals).

Goals

Business Goals

- Accumulate a structured, ML-ready historical dataset of requests, PI profiles, enrichment data, and ROI outcomes to enable a future predictive scoring model that reduces manual enrichment dependency for new requests.
- Reduce the time data managers spend manually evaluating and triaging requests by at least 50%.
- Ensure that highest-impact research, as measured by research output and funding, consistently receives service priority.
- Establish a defensible, auditable methodology for request prioritization that can be communicated to leadership.
- Build institutional knowledge over time about private investigator research activities and outputs.

User Goals

- Data managers can open the tool daily and immediately view a ranked, up-to-date requests queue.
- Users can adjust scoring weights at the metric category level (e.g., Publication Impact, Funding Activity) to reflect team priorities.
- Users can drill down to see underlying data supporting each score, allowing for informed manual overrides.

- The tool provides contextual insights by enabling users to compare new requests against historical PI and request data.

Non-Goals

- This tool does not automate or make final fulfillment decisions; it exists to inform, not replace human judgment.
 - It does not replace REDCap as the system of record for service requests.
 - The tool never exposes scores or rankings to the requesting researchers (PIs or general faculty).
-

User Stories

Persona 1: Data Manager (Primary)

- As a Data Manager, I want to upload the latest REDCap Excel file with a single click and see a ranked queue of all incoming EHR requests, so I can quickly identify which requests to address first.
- As a Data Manager, I want to click on any request to see the detailed breakdown of its score, so that I can understand why it is ranked where it is.
- As a Data Manager, I want to be able to manually override a request's rank with a required note, so I can document exceptions (e.g., urgent deadlines or missing information).
- As a Data Manager, I want to compare new requests to similar past requests by topic and easily view all historical requests by the same PI, so I can leverage institutional knowledge.
- As a Data Manager, I want to upload the Trac hours file alongside the REDCap file so that the tool can calculate the true effort-adjusted ROI score for each request, helping me understand not just which researchers are high-impact but which requests represent the best return on my team's time.
- As a Data Manager, I want to download a minimal Excel export of all open requests ranked by ROI score, so I can share prioritized request status with my team in meetings without exposing full scoring details.

Persona 2: Team Lead/Admin (Secondary)

- As a Team Lead, I want to adjust the weighting of the four main ROI score categories via a configuration panel, so I can reflect the current organizational priorities in triage (e.g., emphasizing funding during grant renewal season).
- As a Team Lead, I want to view a complete audit log of all scoring and override actions, so I can review decision rationales over time.
- As a Team Lead, I want to export the current ranked queue and underlying scores to share with my team, so we can discuss priorities in meetings.
- As a Team Lead, I want to receive data-driven suggestions for category weight defaults based on historical outcomes (v2), so that the scoring model continuously improves as institutional data accumulates.
- As a Team Lead, I want to save named weight configuration presets for quick switching between standard and scenario-specific scoring formulas.

- As a Team Lead, I want to preview the effect of weight changes in the scoring dashboard before committing those changes to the live queue.
-

Functional Requirements

Data Ingestion & Parsing (Priority: High)

- REDCap Excel Import: Accept single or batched REDCap Excel exports via drag-and-drop or file selector.
- Trac Excel Import: Accept and parse the Trac Excel export; associate team effort hours with REDCap request IDs.
- Data Persistence: All uploaded REDCap and Trac data is stored persistently in the application database. Data managers do not need to re-upload files on each login. The ranked queue from the most recent upload is always available when a user logs in, accompanied by a visible timestamp showing when the data was last updated (e.g., "Queue last updated: April 3, 2026"). A new upload is only required when there are new or changed requests to process.
- Flexible Single-File Upload: Neither file is strictly required on every upload. The system supports three upload scenarios:
- Incremental REDCap Upload: When a new REDCap file is uploaded, the system identifies net-new request records (by Record ID) and appends them to the existing database. It does not overwrite or duplicate existing records. Records present in both the existing database and the new upload are checked for status changes (e.g., a request moving from open to closed) and updated accordingly.
- Validation: Auto-detect and flag missing/ambiguous fields, corrupted Excel serials, or format inconsistencies.
- Zero-Friction Upload: Immediate, friendly feedback on upload/parse status; error detection per file; does not block data import for minor issues.
- REDCap Excel Import: Accept single or batched REDCap Excel exports via drag-and-drop or file selector. The 'date_complete' field is a required parsed field used to determine open vs. closed status for all requests. Null value = open; any date value = closed. This field drives queue filtering, the open request export, and PI Profile completion rate calculations.
- Trac Excel Import: Accept and parse the Trac Excel export; associate team effort hours with REDCap request IDs.
- Data Persistence: All uploaded REDCap and Trac data is stored persistently in the application database. Data managers do not need to re-upload files on each login. The ranked queue from the most recent upload is always available when a user logs in, accompanied by a visible timestamp showing when the data was last updated (e.g., "Queue last updated: April 3, 2026"). A new upload is only required when there are new or changed requests to process.
- Flexible Single-File Upload: Neither file is strictly required on every upload. The system supports three upload scenarios: REDCap + Trac (both files): Full scoring is performed. Both Weighted Return Score and Effort-Adjusted ROI Score are computed using actual Trac hours. REDCap only (no Trac): REDCap data is parsed and scored. The Effort-Adjusted ROI Score is computed using

- request-type historical hour estimates from the existing Trac cache in the database. A visible indicator notes that effort hours are estimated, not from a current Trac upload. Trac only (no REDCap): The new Trac data is parsed and merged with existing REDCap records already stored in the database. Hours are updated for matching REDCap IDs. Scores are recomputed for affected requests. This supports mid-cycle Trac updates without requiring a full REDCap re-upload.
- Incremental REDCap Upload: When a new REDCap file is uploaded, the system identifies net-new request records (by Record ID) and appends them to the existing database. It does not overwrite or duplicate existing records. Records present in both the existing database and the new upload are checked for status changes (e.g., a request moving from open to closed) and updated accordingly.
 - Validation: Auto-detect and flag missing/ambiguous fields, corrupted Excel serials, or format inconsistencies.
 - Zero-Friction Upload: Immediate, friendly feedback on upload/parse status; error detection per file; does not block data import for minor issues.
 - Flexible Single-File Upload: Neither file is strictly required on every upload. The system supports three upload scenarios:
 1. REDCap + Trac (both files): Full scoring is performed. Both Weighted Return Score and Effort-Adjusted ROI Score are computed using actual Trac hours.
 2. REDCap only (no Trac): REDCap data is parsed and scored. The Effort-Adjusted ROI Score is computed using request-type historical hour estimates from the existing Trac cache in the database. A visible indicator notes that effort hours are estimated, not from a current Trac upload.
 3. Trac only (no REDCap): The new Trac data is parsed and merged with existing REDCap records already stored in the database. Hours are updated for matching REDCap IDs. Scores are recomputed for affected requests. This supports mid-cycle Trac updates without requiring a full REDCap re-upload.
 - Incremental REDCap Upload: When a new REDCap file is uploaded, the system identifies net-new request records (by Record ID) and appends them to the existing database. It does not overwrite or duplicate existing records. Records present in both the existing database and the new upload are checked for status changes (e.g., a request moving from open to closed) and updated accordingly.
 - Validation: Auto-detect and flag missing/ambiguous fields, corrupted Excel serials, or format inconsistencies.
 - Zero-Friction Upload: Immediate, friendly feedback on upload/parse status; error detection per file; does not block data import for minor issues.

API Enrichment Engine (Priority: High)

- Scheduled enrichment cycle (e.g., every 14 days or set by Admin).
- Google Scholar Metrics: Retrieve h5-index and h5-median (journal-level only) via public API or supported libraries.
- iCite: Pull RCR, NIH percentile, citation counts, and author counts per publication for PIs.
- Clarivate JCR: Retrieve Impact Factor and Q1-Q4 quartile for each journal based on ISSN.

- NIH RePORTER: Fetch active grant count, current award amounts, and renewal/follow-on status for PIs.
- WHO RePORT: Supplement as needed for international funding and trials.
- PI Enrichment Cache: All API-returned data for each PI (citation metrics, grant data, journal prestige metrics) is stored in a persistent PI enrichment cache keyed by PI identifier (last name + department + Wake Forest affiliation). The cache is timestamped with the date of the last successful enrichment run. Scoring always reads from this cache — never from live API calls at upload time. When the scheduled enrichment job runs (14-day or monthly), it updates the cache for all PIs in the system. The cache timestamp is displayed in the UI so data managers know how current the enrichment data is.
- Retry/Failure Handling: All enrichment jobs log failures and attempt config-based automatic retry on next scheduled cycle. Amber-flag for manual review.

PubMed Custom API Integration (Priority: High)

- Purpose: Retrieve all published outputs by PI, minimize false positives via author/affiliation filter.
- Query Construction: Smart keyword extraction for project title and description; stopword removal; no hardcoded domain keyword lists.
- Keyword Extraction Strategy: Adaptable extraction to maximize recall for rare/niche topics; configurable if needed for complex queries.
- Result Fetching & Filtering: Use NCBI esearch + efetch; fetch max 200 PMIDs per PI, with robust affiliation filter for Wake Forest.
- Within-Request Deduplication: Ensure unique PMIDs per request and per PI for scoring.
- Cross-Request PMID Handling: Maintain many-to-many mapping of PMIDs to request IDs to avoid double counting.
- Output: Store all relevant publication metadata, including authorship order, author count, year, and DOI.
- Rate Limiting: Respect and adapt to NCBI limits, including exponential backoff.

ROI Scoring Engine (Priority: High)

- **Score Computation:**
 - *Publication Impact* (internal weights fixed)
 - Mean RCR (70%): Average Relative Citation Rate across all PI publications (iCite).
 - Average Author Count (30%): Mean authors per PI publication (iCite), min-max normalized; higher is better.
 - Citation count, NIH percentile, first/last author proportion (iCite/PubMed): Informational only, displayed in PI Profile but not scored.
 - *Journal Prestige* (internal weights fixed)
 - Impact Factor (30%): Clarivate JCR.
 - JCR Quartile (30%): Linear mapping, Q1=100, Q2=66, Q3=33, Q4=0 (Clarivate JCR).

- h5-index (20%): Google Scholar Metrics.
 - h5-median (20%): Google Scholar Metrics.
 - Average across all matched journals for PI, weighted by number of publications per journal.
 - **Niche Journal Bonus:** 1.10–1.15x multiplier if specialty Mode < \$50B AND CAGR > 7%, thresholds and multiplier configurable in admin panel.
- *Funding Activity* (internal weights fixed)
 - Total Award Amount (50%): NIH/WHO RePORTER.
 - Active Grant Count (30%): NIH/WHO RePORTER.
 - Follow-On Grant Boolean (20%): 1/0 if PI has received competitive renewal; default 0 if data not available.
- *Research Output* (internal weights fixed)
 - Publication volume (50%): 5-year rolling unique publication count (PubMed).
 - Collaborative network size (50%): Unique co-authors across all PI publications (PubMed).
- **Weighted Return Score Formula:**
 Weighted Return Score = (w1 × Publication Impact) + (w2 × Journal Prestige) + (w3 × Funding Activity) + (w4 × Research Output)
 (w1-w4 sum to 1.0, displayed as percentages; default w1=w2=w3=w4=0.25)
- **Effort-Adjusted ROI Score Formula:**
 Effort Multiplier = request total_hours / team average total_hours;
 Effort-Adjusted ROI Score = Weighted Return Score / Effort Multiplier, rescaled 0–100.
- **Score Normalization:** Min-max scaling for categories and sub-metrics; dynamic rescaling post effort division.
- **ML Readiness:** All scores, intermediate metrics, and binary outcomes (is_published, is_completed) stored for future model training.
- **ML Feature Engineering Note:** Boolean/binary feature derivation (completion status, publication outcome, etc.) is developer's responsibility during ML model prep—raw data must permit this.

Ranking Dashboard (Priority: High)

Ranking Dashboard (Priority: High)

- Ranked Queue UI: Immediate, sortable ranked queue of requests by Effort-Adjusted ROI Score (default).
- Score Breakdown: Expandable/collapsible data on each request with full underlying metrics and API sources.
- Manual Override: Enable rank override with reason logging; audit trail stores user, time, and rationale.
- Dual Score Columns: Show both Weighted Return Score and Effort-Adjusted ROI Score clearly; allow toggling sort.

- **Open Request Export:** A dedicated export function allows any user (Data Manager or Team Lead roles) to download the current open request queue as a formatted Excel (.xlsx) file. Open vs. closed determination: a request is considered open if the REDCap 'date_complete' field is null and closed if 'date_complete' contains any date value; this logic is applied consistently across the tool (no manual status flagging required). The export includes only open requests (date_complete is null) sorted by Effort-Adjusted ROI Score descending (default), matching the live ranked queue default. The exported Excel contains only these columns in this order: Rank, PI Name, Department, Funding Amount, Study Title, Request Type, Effort-Adjusted ROI Score. The file name is auto-generated with the export date (e.g., "Open_Requests_2026-04-05.xlsx"). The export button is accessible directly from the main ranked queue dashboard (no navigation required). The export reflects the queue state at the moment of download — including any manual overrides that have been applied; override-adjusted ranks are reflected in the exported Rank column.

Weight Configuration Panel (Priority: Medium)

- **Default Weights:** 25% per category; cannot drop below 5%.
- **Slider Interface:** Four main sliders, one per category, with auto-normalization ensuring sum=100%.
- **Auto-Normalization:** Adjusting any slider renormalizes others proportionally.
- **Minimum Weight Floor:** No category below 5%.
- **Metric-level v2 note:** Per-metric controls not present in v1, but documented for future expansion if needed.
- **Live Preview:** Real-time queue reordering, labeled as preview until “Apply” is pressed.
- **Apply Button:** Changes only commit on Apply, not instantaneously.
- **Named Preset System:** Save/load named weight configs; always-available undeletable “Default (Equal Weights)” preset.
- **Preset Audit Log:** All apply/preset changes timestamped and attributed in audit trail.

Medical Specialty Classification & Market Sizing (Priority: Medium)

- **Purpose:** Map each journal to medical specialty, then contextualize with Mode market size, CAGR, and related estimates.
- **Journal Resolution Pipeline:** Three-tier matching:
 1. Exact ISSN match via CrossRef
 2. Normalized title match
 3. Fuzzy title match ($\geq 85\%$ similarity, else flag as “Unclassified”)
- **CrossRef Integration:** Resolve PMIDs to ISSN and full title (use PubMed XML as fallback).
- **Primary Specialty Determination:** Assign PI the most frequently matched specialty; display others as side notes.
- **Market Size Display:** Display Mode as headline; min/max/mean/CAGR/Confidence/Source in tooltip/side panel.

- Niche Market Side Note: If primary specialty qualifies as niche, display market context beside request.
- Average Publications Per Specialty Per Year: Reference static table (refreshed annually) using ISSN-to-PubMed query aggregation.
- Dataset Maintenance Note: Specialty/ISSN and market size data sets are statically maintained; manual refresh 1–2x per year.

Request Similarity Matching (Priority: Medium)

- Topic-Based Similarity: Keyword/topic/dept/title-based matching; highlights past requests for contextual review.
- PI History View: On request drill-down, show 9 PI Profile data points, including prior requests, completion/publication rates, unique pubs, rank, funding, collaboration, and request-type history.
- PMID Deduplication Logic: Ensure Within-request and Cross-request PMIDs are unique/aggregated as appropriate.

Historical Context (Priority: Medium)

- Prior Request Tracking: Show previous requests by same PI/faculty, with outcomes and completion status for context.
-

User Experience

Entry Point & First-Time User Experience

- Data managers access the secure internal web app via Wake Forest SOM network credentials.
- First login offers a brief onboarding and dashboard walkthrough.
- On subsequent logins (not first-time), the data manager lands directly on the ranked queue populated with the most recently uploaded and scored data. A timestamp shows when the queue was last updated and when enrichment data was last refreshed. No upload is required to view the existing queue.
- If no data has been uploaded, user is clearly prompted to upload or connect to latest REDCap Excel file.

Core Experience

1. Step 1 — Upload (when new data is available): The upload interface accepts two files — REDCap Excel and Trac Excel — via drag-and-drop or browse. Neither file is required on every session. The interface presents three clearly labeled upload options: (a) Upload both files, (b) Upload REDCap only, (c) Upload Trac only. Each option shows a brief description of what will happen (e.g., 'Upload Trac only — updates hours for existing requests and recomputes scores'). If no upload is needed, the data manager can dismiss the upload prompt and proceed directly to the existing ranked queue. File validation runs independently per file. Missing or ambiguous fields are flagged per file. Incremental REDCap uploads

- append new records and update status changes without overwriting existing data.
2. Parse: Interface validates, parses, and flags errors. Proceeds if only minor parsing issues.
 3. Enrich & Score: System enriches with API data; scores computed using most recent enrichment results. Requests are visually flagged for missing data.
 4. Rank: Presents a sortable, dual-score (Return/ROI) queue. Invalid or missing Trac hours are highlighted in amber with explanation.
 5. Drilldown: Clicking a request opens full breakdown: score subs, publication/journal/funding/output, and enrichment sources.
 6. Context: Similar requests (topic, PI, dept.) and PI Profile are available in side panel with all 9 defined PI stats.
 7. Override: Data manager may override rank (must supply reason); changes are fully audit-logged.
 8. Step 8 — Export: Two export options are available from the dashboard:
 - Full Queue Export: Exports the complete ranked queue (open and closed requests) with all score details for offline review or leadership reporting.
 - Open Requests Export: Exports only open requests (date_complete is null) as a minimal Excel file containing: Rank, PI Name, Department, Funding Amount, Study Title, Request Type, and Effort-Adjusted ROI Score. Sorted by Effort-Adjusted ROI Score descending. File is auto-named with the export date (e.g., "Open_Requests_2026-04-05.xlsx"). Manual overrides are reflected in the exported rank. This export is designed for team meetings and daily status reviews — minimal, scannable, and immediately shareable.
 9. Export: Team Lead/data manager can export current queue with all scoring data for agendas or records.

Advanced Features & Edge Cases

- Only Team Leads can access weight config panel and apply changes.
- Requests lacking PI/IRB are clearly flagged and removed from auto-ranking.
- Robust PI disambiguation for common names (using dept/inst context).
- Automated, background enrichment job scheduler.
- Timely failure alert/retries; clear amber-flag for enrichments/manual review.
- *Audit log captures all user activity, overrides, and config updates.*
- Logical multi-column, multi-format input handling.
- Responsive UI and clear, minimal workflow.

UI/UX Highlights

- Visual charts (bar/stack) breaking down scores.
- “Similar Requests” panel in all detail views.
- PI Profile access at queue or request level.
- Confirmation modals for significant actions (e.g., override, apply weights).
- Missing/partial enrichment data visually flagged.

- Accessible and performant for expected traffic.
-

Narrative

Alex, a data manager for the Wake Forest School of Medicine, arrives at work to process EHR data requests. Now, instead of manually researching each principal investigator and project, Alex logs in, uploads the latest REDCap export, and immediately receives a sorted, ranked queue of requests. Every score leverages the most up-to-date API-enriched PI data, refreshed regularly behind the scenes. For each request, Alex can quickly see detailed ROI metrics—including PI's publication impact, key journal prestige from Clarivate JCR, and current funding activity drawn from NIH RePORTER and WHO RePORT.

If Alex needs more context, they use the built-in topic similarity tool, surfacing past requests in the same department or on similar research topics, helping anticipate challenges or outcomes. When a familiar PI appears, Alex uses the PI History view to audit all interactions—a full institutional memory at a glance. When a judgment call requires override, Alex documents the rationale, ensuring transparency with downstream reviewers. All scoring assumptions remain visible in the admin panel, where the Team Lead easily steers triage with a few intuitive sliders. The result: Alex and the team operate at maximum strategic value, prioritizing the highest-ROI research with minimal manual effort and seamless institutional continuity.

Success Metrics

User-Centric Metrics

- ≥80% of data managers use the ranked queue as primary triage within 3 months.
- 50% reduction in average triage time per request batch (vs. pre-tool baseline).
- Internal user satisfaction survey ≥4 out of 5.

Business Metrics

- Increase in completed requests attributed to federally-funded or high-impact projects (tracked quarterly).
- Reduction in escalations or complaints regarding triage transparency.

Technical Metrics

85% API enrichment data pull success rate per request.

- <2 minutes score calculation time for batches of 20 requests, using most recent enrichment data.
- ≥99% dashboard uptime during core business hours.

Tracking Plan

- Log each REDCap upload/import event.
- Log each scheduled API enrichment cycle (success/failure by provider).

- Audit trail of every manual override (note, user, timestamp).
 - Audit all weight configuration changes.
 - Track daily active users (logins or dashboard opens).
-

Technical Considerations

- Excel/CSV file parser for REDCap data (29 columns), tolerant of minor format changes.
- Trac Excel parser with multi-ticket hour summation per REDCap ID; corrupted date-serial detection and flagging; request-type-based average hour lookup for new request estimation; effort multiplier calculation engine using team average hours as baseline.
- API integration for Google Scholar, iCite, NIH RePORTER, Clarivate JCR (institutional subscription), WHO RePORT.
- Custom PubMed API integration using NCBI E-utilities (esearch + efetch); query construction using stopword-based keyword extraction (no hardcoded domain lists); many-to-many junction table for PMID-to-request relationships; DISTINCT PMID aggregation for PI Profile publication counts.
- ROI scoring engine using category weights and effort-normalization.
- Internal web app: dashboard, scoring details, similarity context, admin configuration.
- Database for storing requests, scores, PI enrichment cache, override records, logs.
- PI name disambiguation using department/institution context.
- Text/keyword-matching engine for topic-based similarity analysis.
- Niche journal bonus multiplier engine using Medical Specialty Market Sizing dataset thresholds (Mode market size and CAGR); admin config panel for threshold and multiplier value adjustment without code changes; JCR quartile-to-score linear mapping (Q1=100, Q2=66, Q3=33, Q4=0); 5-year rolling publication window filter for Research Output.
- Persistent application database storing all REDCap request records, Trac hour totals, PI enrichment cache (keyed by PI identifier, timestamped), scores, overrides, audit logs, and weight presets. Incremental REDCap record processing (append new, update changed, preserve existing). Trac-only upload merge logic matching Trac tickets to existing REDCap records by Record ID. Session persistence — ranked queue always available on login without re-upload.

Technical Needs

Technical Needs

- Integration with all specified APIs (iCite, Google Scholar, Clarivate JCR, NIH RePORTER, WHO RePORT, PubMed/NCBI).
- Secure file upload with error catching and feedback to user.
- Scheduled background enrichment services (not blocking UI).

- Relational DB model to handle many-to-many PMIDs/requests.
- Configurable admin panel for scoring weights, thresholds, and bonuses.
- Audit logging and historical config versioning.
- Excel (.xlsx) export engine for open request queue; date_complete null-check logic for open/closed request filtering; auto-generated filename with export date; override-aware rank column reflecting any manual rank adjustments.

Integration Points

- REDCap: manual Excel/CSV export (no live API needed for v1).
- iCite, NIH RePORTER, WHO RePORT (public REST APIs).
- Google Scholar (via supported 3rd-party libraries/APIs).
- Clarivate JCR (full API access, institutional).
- All enrichment jobs run on a background schedule, independent of uploads.

Data Storage & Privacy

- No patient-level data is handled; only PI, faculty, study, and grant metadata.
- Access-controlled application, audit-logged user actions.
- Cached enrichment data always timestamped and sourced.
- All uploaded REDCap and Trac data is retained in the application database between sessions. The PI enrichment cache is persistent and updated on the scheduled enrichment cycle. Users are never required to re-upload data to access a previously generated ranked queue. Data retention policy should be defined by the Wake Forest SOM IT and compliance team.

Scalability & Performance

- Designed for 20–50 daily requests; API enrichment is batched and scheduled for performance.
- Upload-triggered scoring is always performed against latest cached PI enrichment data.
- No complex scaling required; high responsiveness for small team.
- ML model training requires a minimum corpus of completed requests with known outcomes (recommended: 200+ records with is_completed and is_published flags populated). The team should monitor dataset growth and plan ML readiness review accordingly. Early ML attempts on insufficient data will produce unreliable models.
- PI name disambiguation for common names—resolve with department/institution context.
- Some external APIs (e.g., Google Scholar) require rate management and failure handling.
- Clarivate JCR API quotas handled; institutional access confirmed.
- Similarity matching requires robust text parsing and normalization.
- PMID-to-request relationships are many-to-many — one publication can result from multiple data requests by the same PI. The data model must support this

relationship explicitly, and all publication count metrics in the PI Profile must aggregate on unique PMIDs to avoid inflation.

- Trac data quality is inconsistent — corrupted Excel date serial values must be detected programmatically (range check ~40000–50000) and defaulted gracefully. Multi-ticket summation per REDCap ID must be handled before effort score computation. Request-type average hours must be recomputed periodically as new Trac data is added.
 - Follow-on grant detection relies on NIH RePORTER data availability and grant naming conventions — reliable identification of competitive renewals may require pattern matching on grant IDs or titles and should be validated against known renewal records during development. When unavailable, defaults to 0 and is flagged.
 - Incremental REDCap uploads must correctly identify new vs. existing records by Record ID and handle status changes without data loss. Trac-only uploads must accurately match ticket-level records to existing REDCap Record IDs — unmatched Trac tickets should be flagged for review rather than silently discarded.
 - The date_complete field must be consistently populated in REDCap exports — if the field is missing or misnamed across export versions, open/closed filtering will fail silently. The parser should validate the presence of date_complete on every upload and alert if the field is absent.
-

Milestones & Sequencing

Project Estimate

- **Medium:** 3–4 weeks to MVP

Team Size & Composition

- **Small Team:** (3 people)
 - 1 Backend Developer (parsing, enrichment, scoring engine)
 - 1 Frontend Developer (dashboard, UI/UX)
 - 1 Product/Data Manager Stakeholder (requirements, UAT, coordination)

Suggested Phases

Phase 1 – Core Pipeline (Week 1–2)

Phase 1 – Core Pipeline (Week 1–2)

- Implement REDCap Excel parser and data model.
- Parse Trac Excel file, implement multi-ticket hour summation per REDCap ID, detect and flag corrupted hour values.
- Implement request-type average hour estimation for new requests.
- Build scheduled API enrichment jobs for PI-level data (iCite, NIH RePORTER, Clarivate JCR, WHO RePORT, Google Scholar).

- Build custom PubMed API integration with stopword-based keyword extraction, affiliation filter, post-fetch filtering, within-request deduplication, and many-to-many PMID data model.
- Develop scoring engine with configurable, grouped category weights and effort multiplier.
- Implement min-max normalization engine.
- Design ML-ready feature storage schema.
- Enable basic similarity matching engine (topic/dept/title).
- Generate preliminary ranked queue; Excel output is sufficient at this stage.
- Run historical PubMed publication backfill script for all closed REDCap requests (date_complete is not null) at deployment — populates PI Profile publication rate and publication count from existing historical data using the same PubMed query and deduplication logic as the ongoing integration.

Phase 2 – Dashboard & Scoring UI (Week 2–3)

- Build web dashboard for ranked queue and ROI breakdown.
- Implement dual score display (Weighted Return Score + Effort-Adjusted ROI Score) in ranked queue and request detail view; display corrupted/missing Trac values in amber with tooltip.
- Enable manual override with notation and audit logging.
- Implement "Similar Requests" and "PI History" features in request detail view.
- User authentication and admin panel for scoring weights.
- Integrate CrossRef API, implement three-tier journal-to-specialty matching, build specialty and market size display.

Phase 3 – Full Hardening & Usability (Week 3–4)

- Strengthen PI disambiguation logic.
- Ensure robust file parsing, error handling, and security.
- Finalize UI polish, logs, and export functions.
- QA and UAT; ready for pilot rollout and feedback.
- Validate follow-on grant detection logic against known NIH RePORTER renewal records.
- Configure and test niche journal bonus multiplier against specialty market sizing thresholds.
- Validate JCR quartile mapping and journal prestige averaging across multi-journal PIs.

V2 Enhancements & ML Opportunities

The v1 tool is explicitly designed as an ML-ready foundation. All raw API values, normalized subscores, category weights, effort hours, request metadata, binary outcome flags (is_completed, is_published), and final ROI scores are stored in a structured schema to support a future supervised machine learning model. The following v2 opportunities are documented for the development team and product stakeholders:

1. **Predictive ROI Scoring (Core ML Model):**
Once sufficient historical data has accumulated (recommended minimum: 12–18 months of completed requests with known outcomes), a supervised ML model can be trained to predict the likely ROI score for a new incoming request before any enrichment data is available. Input features include: PI faculty rank, department, funding source type, request type, funding mechanism, and any available PI history. Target variable: Effort-Adjusted ROI Score from historical completed requests. This model would surface a "Predicted ROI" estimate for brand-new PIs or first-time requestors where enrichment data is sparse. Binary derived features required for model training (is_completed: true/false, is_published: true/false) are not UI features — they are derived by the developer from raw stored data during ML dataset preparation.
2. **ML-Informed Default Category Weights:**
The v1 tool initializes with equal category weights (25/25/25/25) set by judgment. In v2, ML can analyze which category signals most strongly predict publication rate and completion rate historically, and surface data-driven suggested default weights to the team lead. For example, if Funding Activity consistently shows the strongest correlation with publication outcomes, the ML model can recommend a higher default weight for that category. The team lead retains full manual control — ML suggestions are advisory, not automatic.
3. **ML-Informed Default Metric Weights:**
The v1 tool uses fixed internal metric weights set by judgment (e.g., Mean RCR 70% / Average Author Count 30% within Publication Impact). In v2, ML can learn optimal metric-level weights from historical outcome data and recommend calibrated defaults. This directly enables the metric-level weight control feature documented as a v2 enhancement in the Weight Configuration Panel — ML informs the starting point, the team lead adjusts from there.
4. **ML-Improved Effort Hour Estimation:**
The v1 tool estimates hours for new requests using the historical average for that request type. In v2, a regression model can predict effort hours more precisely using a richer feature set: request type, PI faculty rank, department, funding type, data complexity indicators from REDCap, and PI historical average hours. This improves the accuracy of the Effort Multiplier for new requests, making the Effort-Adjusted ROI Score more reliable from day one.
5. **ML-Optimized Niche Journal Bonus Thresholds:**
The v1 niche journal bonus uses manually set thresholds (\$50B market size Mode, 7% CAGR) configurable via admin panel. In v2, ML can analyze which market size and CAGR combinations most strongly correlate with high-ROI outcomes in the historical dataset and recommend optimal threshold values. The team lead retains the ability to override ML-suggested thresholds.
6. **Semantic Topic Similarity Matching:**
The v1 topic similarity feature uses keyword and text-based matching on department, project title, and study description. In v2, semantic embedding models (e.g., sentence transformers or domain-specific biomedical embeddings such as BioBERT) can dramatically improve match quality by capturing conceptual similarity rather than surface keyword overlap. For example, a study on "myocardial infarction outcomes" would correctly match a past study on "cardiac event recovery" even without shared keywords.

7. **ML-Improved PI Disambiguation:**

The v1 PI disambiguation logic uses rule-based matching combining PI last name, department, and Wake Forest affiliation. In v2, a probabilistic or ML-based name disambiguation model can improve confidence scoring for ambiguous cases — particularly for common names where department and affiliation alone are insufficient. This reduces false positive PI matches that could distort enrichment data and scores.

8. **Data Accumulation Milestone:**

The development team should monitor dataset size and flag when the historical request corpus reaches a sufficient size to begin ML model training.

Recommended trigger: 200+ completed requests with known `is_published` and `is_completed` outcomes. At that point, the team lead and product stakeholder should initiate an ML readiness review before beginning v2 model development.

Developer Handoff Callouts

This section consolidates all known caveats, risks, and implementation nuances raised during PRD development. Each item should be reviewed and resolved before or during development. Items marked **[DECISION NEEDED]** require confirmation from the product owner before implementation.

Data Ingestion & Parsing

1. The REDCap Excel export column names may vary slightly across export versions. The parser must be tolerant of minor column name changes and should validate the presence of all required fields on every upload, alerting the user immediately if a required field is absent or renamed. Critical fields to validate: PI name, department, faculty rank, funding source, IRB number, request type, `date_complete`, and Record ID.
2. The `date_complete` field drives all open/closed logic throughout the tool — queue filtering, the open request export, and PI Profile completion rate. If this field is missing or misnamed in a REDCap export, open/closed filtering will fail silently. The parser must explicitly validate and alert on this field on every upload.
3. Trac data contains corrupted hour values that appear as 5-digit Excel date serial numbers (range approximately 40,000–50,000). These must be detected programmatically by range check, flagged visually in amber in the UI, and defaulted to 2 hours. They must not be imported as hour values.
4. Multiple Trac tickets may map to the same REDCap Record ID. All hours across all matching tickets must be summed before computing the effort multiplier for that request. Summation must happen before any scoring computation.
5. Unmatched Trac tickets (Trac entries with no corresponding REDCap Record ID in the database) must be flagged for review rather than silently discarded. A report or UI indicator of unmatched Trac entries should be surfaced to the data manager after each Trac-only upload.
6. Incremental REDCap uploads must correctly identify new vs. existing records by Record ID. New records are appended. Existing records are checked for status changes only (e.g., `date_complete` populated). No existing records should be overwritten or duplicated.

7. Publication rate (proportion of requests that yielded at least one publication) requires that completed requests are linked to PubMed results via the PubMed custom API. A historical backfill job is confirmed in scope for v1. At deployment, the developer will run a one-time backfill script that queries PubMed for all historically closed requests already in the REDCap database (closed = date_complete is not null), populating publication data retroactively for all existing PIs. This ensures the PI Profile publication rate and publication count fields are meaningfully populated from day one rather than starting empty. The backfill script uses the same query construction, filtering, deduplication, and PMID junction table logic as the ongoing PubMed custom API integration. This is a one-time script run at deployment — not a recurring job.

API Enrichment & PubMed Integration

1. Google Scholar has no official API. H5-index and h5-median retrieval depends on third-party scraping libraries, which are fragile and subject to rate limiting and breakage. These metrics must be included in the scheduled 14-day enrichment cycle with retry logic and graceful failure handling. If Google Scholar data is unavailable, the Journal Prestige subscore should proceed using only Clarivate JCR data (Impact Factor + Quartile) with a visible flag that h5 metrics are missing.
2. The PubMed affiliation filter uses "Wake Forest"[Affiliation] as a constant. This is the primary defense against false PI name matches. Confirm with the development team that this affiliation string returns consistent results across PubMed's indexed records — variations like "Wake Forest University" vs. "Wake Forest Baptist Medical Center" vs. "Wake Forest School of Medicine" may need to be OR-combined in the query.
3. The PubMed keyword extraction strategy strips standard English stopwords plus a custom generic research term list (e.g., "study," "analysis," "assessment," "outcomes," "impact," "effect," "among," "following") and selects the two longest remaining tokens. This custom stopword list should be stored as a configurable file — not hardcoded — so it can be updated without a code change.
4. PubMed retrieves up to 100 PMIDs per query. Filtering (year >= Year Completed + 2, PI in author list) is applied post-fetch, not pre-fetch. The final result set is capped after filtering — never before. This ensures relevant papers beyond position 20 are not accidentally excluded.
5. CrossRef API is used to resolve PMIDs to ISSN and full journal name. CrossRef coverage is excellent for recent literature but may be incomplete for older publications. When CrossRef returns no result for a PMID, the PubMed XML journal name field is used as the Tier 2 fallback input for normalized title matching.
6. Follow-on grant detection relies on NIH RePORTER grant naming conventions. Reliable identification of competitive renewals may require pattern matching on grant IDs or titles (e.g., detecting "-A1" or "-R" suffix patterns). This logic should be validated against a set of known renewal records before deployment. When data is unavailable or ambiguous, the follow-on grant field defaults to 0 and is flagged as unverified in the PI Profile.

Scoring Model

1. Min-max normalization is sensitive to outliers. In early deployment when few PIs are in the system, one PI with an unusually high h-index, citation count, or grant amount will compress all other scores toward 0. Consider seeding the normalization with reasonable domain benchmarks (e.g., known Wake Forest faculty ranges) until the dataset is large enough to self-calibrate. Percentile-based normalization is documented as a v2 improvement option.
2. The niche journal bonus multiplier (1.10x–1.15x) is applied to the Journal Prestige subscore and can push that subscore above its normalized ceiling before the effort division and final rescaling step. The rescaling after the effort division must account for this — it should be applied dynamically across the current queue, not as a fixed formula, so relative ordering is preserved correctly.
3. The Effort-Adjusted ROI Score rescaling step must be applied dynamically across the current queue after division. If a request has very low hours (e.g., 0.5x team average), the division will produce a score above 100 before rescaling. The rescaling must preserve relative rank ordering correctly across all requests in the queue.
4. The request-type average hours used for effort estimation on new requests must be recomputed periodically as new Trac data accumulates. This recomputation should be triggered automatically after each Trac upload, not run on a fixed schedule.
5. The follow-on grant boolean defaults to 0 when NIH RePORTER data is unavailable. This means PIs with genuine follow-on grants but missing API data will be systematically underscored on the Funding Activity category. This should be surfaced visually in the PI Profile so data managers can flag it for manual override if needed.

Journal-to-Specialty Matching

1. The ISSN journal classification list (2,322 entries) and Medical Specialty Market Sizing dataset (35 specialties) are both static, manually maintained files. They must be reviewed and updated by the team periodically — recommended annually. No automated refresh is in place. Unclassified journals (failing all three matching tiers) should be logged and surfaced to the team for potential addition to the classification list over time.
2. The three-tier journal matching cascade (ISSN exact → normalized title → fuzzy at 85% threshold) will have an unclassified tail. The team should monitor the unclassified rate after deployment and expand the journal list accordingly. NLP-based matching refinement is documented as a v2 enhancement.
3. PubMed returns journal names as ISO abbreviations (e.g., "J Pediatr Surg"), not full titles. The ISSN journal classification list likely uses full titles. The CrossRef API bridges this gap by returning both ISSN and full title per PMID. Confirm that the CrossRef response is used as the primary journal name input before falling back to PubMed XML, which may return only the ISO abbreviation.

PI Profile & Publication Data

1. Publication rate (proportion of requests that yielded at least one publication) requires that completed requests are linked to PubMed results via the PubMed custom API. This linkage runs post-completion for closed requests. For historical closed requests already in the REDCap database at the time of tool deployment, a backfill job should be run to populate publication data retroactively. **[DECISION NEEDED]** — confirm with product owner whether a historical backfill is in scope for v1 or deferred to post-launch.
2. The average publications per specialty per year reference table is generated by a one-time script querying PubMed for annual publication counts per ISSN, summed across all journals classified under each specialty. This script should be built and run as part of the initial deployment setup and refreshed annually. The output is a static table stored in the database — not a live query at runtime.
3. The PI enrichment cache is keyed by PI identifier (last name + department + Wake Forest affiliation). For PIs with common last names where department alone is insufficient for disambiguation, the cache key may collide across different individuals. The disambiguation logic must be validated against the actual REDCap dataset before deployment to identify any high-risk name/department combinations.

Data Retention & Compliance

1. A data retention policy for all stored REDCap and Trac data should be defined by Wake Forest SOM IT and compliance team before deployment. The PRD does not specify a retention period — this must be confirmed externally. **[DECISION NEEDED]**
2. The tool handles no patient-level data — only PI metadata, study titles, grant information, and publication records drawn from public sources. However, the IRB number field from REDCap is stored in the database. Confirm with compliance whether storage of IRB numbers in this internal tool requires any additional access controls or documentation beyond standard role-based access.

General Developer Notes

1. The metric-level weight controls (sub-sliders within each of the four scoring categories) are explicitly a v2 feature. Do not implement in v1. Internal metric weights (70/30 for Publication Impact, 60/40 split for Journal Prestige, 50/30/20 for Funding Activity, 50/50 for Research Output) are fixed in code by the developer and are not user-configurable in this version.
2. The "Default (Equal Weights)" preset in the Weight Configuration Panel must be permanent and undeletable. All other presets can be renamed or deleted by the Team Lead.
3. All Apply actions in the Weight Configuration Panel must be logged to the audit trail with the full before-and-after weight configuration, not just the changed value. This is required for the scoring model to be auditable and defensible over time.
4. The ML-ready feature storage schema should be designed before any other database tables are built, as it affects the structure of nearly every other table in the system. Specifically: all raw API values, normalized subscores, effort hours, category weights at time of scoring, and binary outcome fields (is_completed,

is_published) must be stored as discrete, queryable fields — not serialized blobs — to support future ML feature extraction without a data migration.

5. Schedule a developer walkthrough specifically covering: (a) the two-level scoring formula and its hierarchical computation order, (b) the effort multiplier rescaling behavior, (c) the many-to-many PMID junction table, and (d) the niche journal bonus interaction with the final rescaling step. These four areas are the most likely sources of implementation error.

Pending: IT & Compliance Question

The following question must be submitted to Wake Forest SOM IT and/or the IRB/compliance office before deployment. The answer will determine the data retention policy and any additional access control requirements for this tool.

Question to submit:

"We are building an internal tool that stores REDCap research request metadata (PI name, department, study title, IRB number, funding source) and team hour logs from Trac. No patient data is stored. What is the required data retention period for these records, and are there any access control requirements specific to storage of IRB numbers in an internal application?"

Once a response is received, update Developer Handoff Callout item 25 (Data Retention & Compliance) with the confirmed policy and provide it to the development team before database schema finalization.